



THE UNIVERSITY OF
MELBOURNE

SWEN90010 - High Integrity Systems Engineering

Logic Intro

Toby Murray



toby.murray@unimelb.edu.au



Room 2107, Level 2, Melbourne Connect



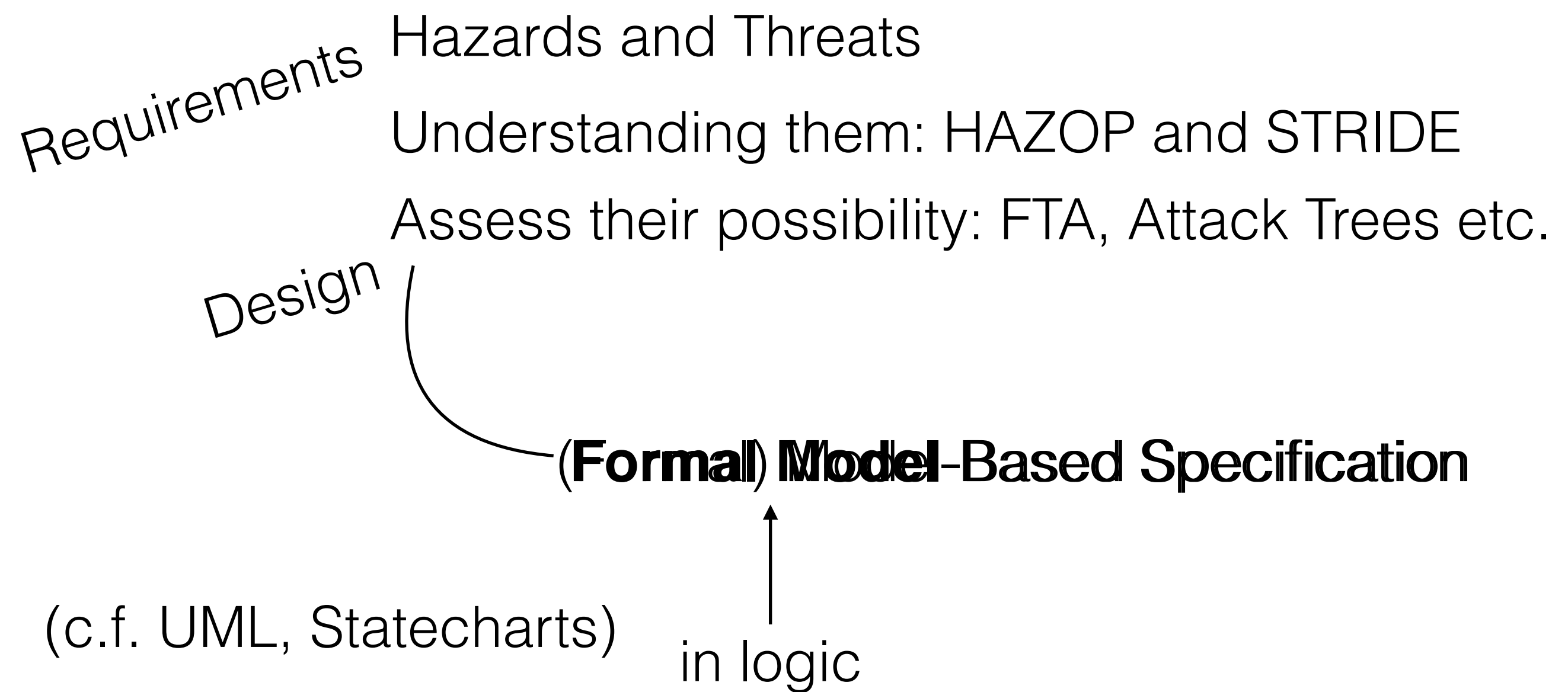
<http://people.eng.unimelb.edu.au/tobym>



@tobycmurray

Story so far

Safety and Security Engineering



Why Logic at all?

Because natural language is ambiguous

Natural language doesn't directly permit logical reasoning
(e.g. to check consistency or properties)

Examples:

“The system must ensure health data is shared securely and
only with authorized parties”

“Only doctors have access to patient medical records.”

Logical Specifications



First we need a **logic**

This course: (first order) set theory / predicate logic

Then, we want a **mechanised** way to **reason** in the logic (i.e. prove things about our specs)

This course: Alloy

Today

Refresher / Introduction to logic

Propositions

Sets

Predicates

Relations

Plus Alloy's flavour of these.

Propositions

A statement that is either true or false

“it is raining”

Raining

atom

“it is raining in Parkville”

Raining[Parkville]

predicate

“it is raining in Parkville
and it is not raining in Sydney”

compound proposition

Propositional Connectives



THE UNIVERSITY OF
MELBOURNE

Traditional Notation

and

$\wedge$

or

$\vee$

not

$\neg$

implies

$\Rightarrow$ (or $\rightarrow$)

iff

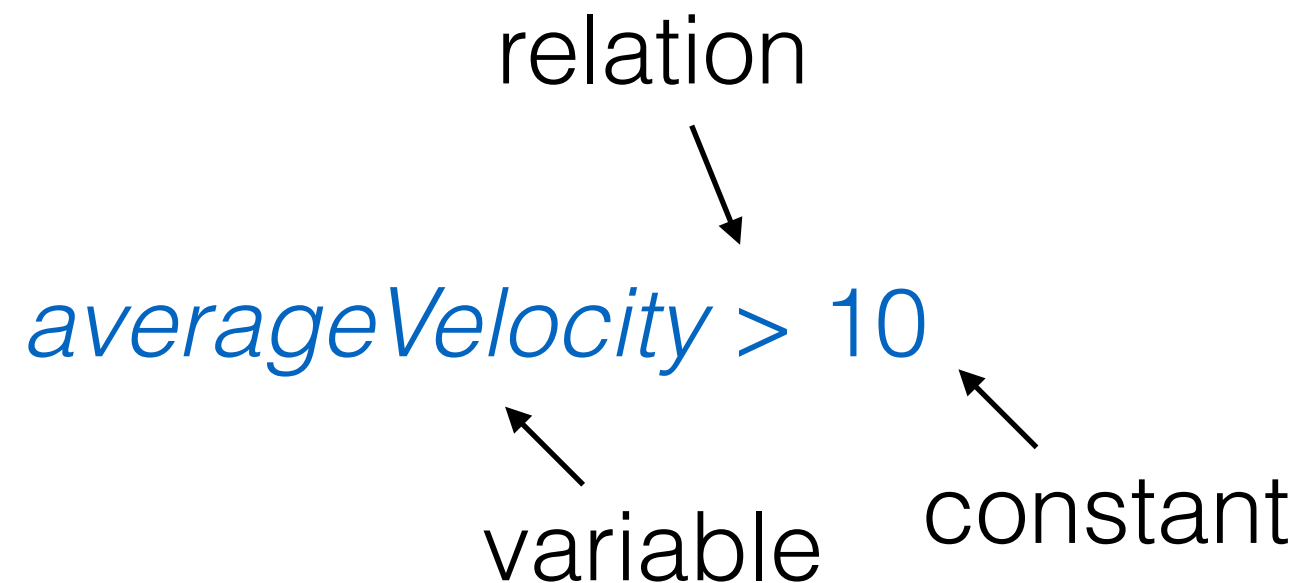
$\Leftrightarrow$

Raining[Parkville] **and not** Raining[Sydney]

Raining[Parkville] **implies not** Raining[Sydney]

Variables

Variables allow propositions to talk about **state**



averageVelocity > 10 **and** *currentVelocity* = 11

Variables talk about different parts of the state of the formal model. (**not** state of the system/program)

Sets



THE UNIVERSITY OF
MELBOURNE

A collection of things

“the set containing 3 and 5”

$\{3, 5\}$

“the set of even numbers”

$\{x \mid \text{even}[x]\}$

universe

the set holding everything

univ

empty set

the set of nothing

none

union

everything in A and everything in B

$A + B$

intersection

everything that's in **both** A and B

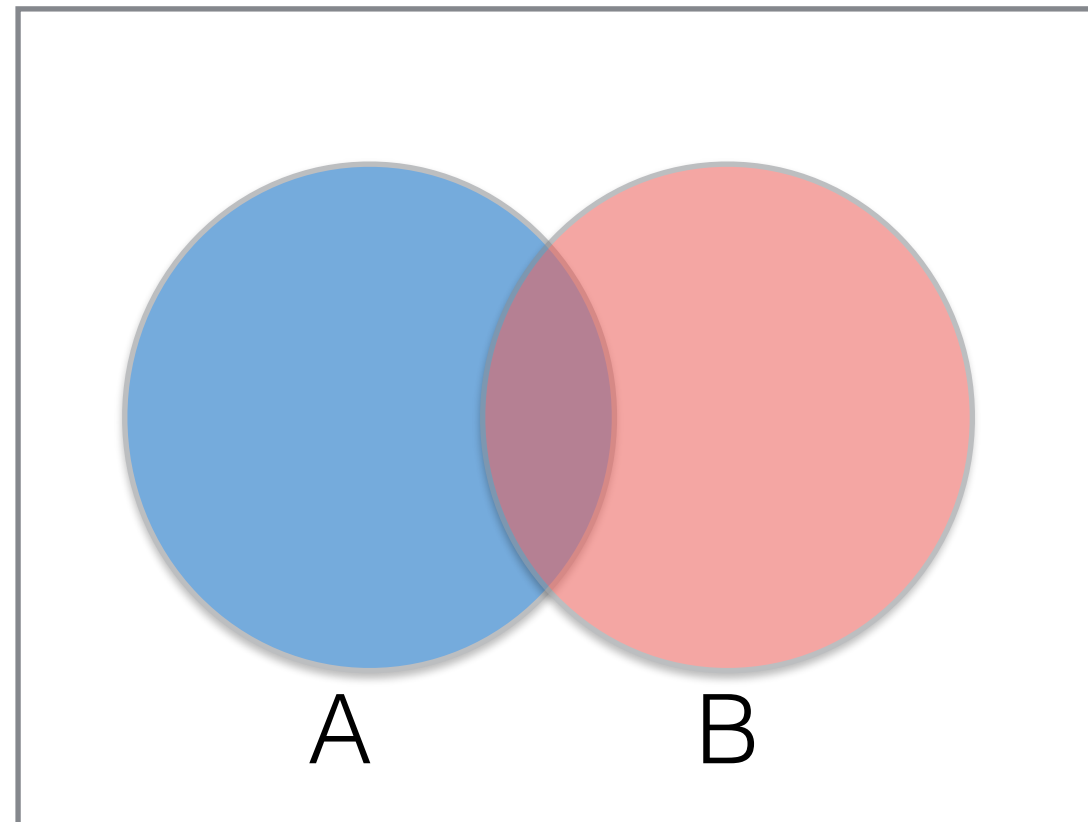
$A \& B$

difference

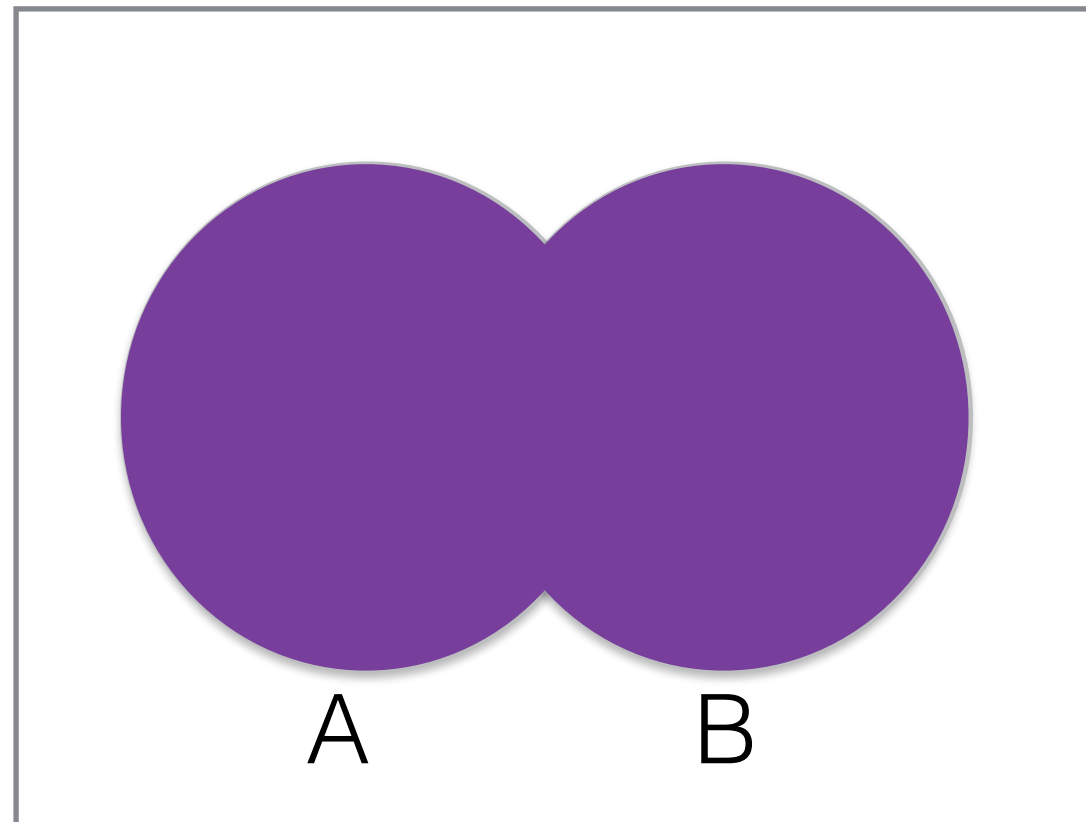
everything in B that is not in A

$B - A$

Set Operations

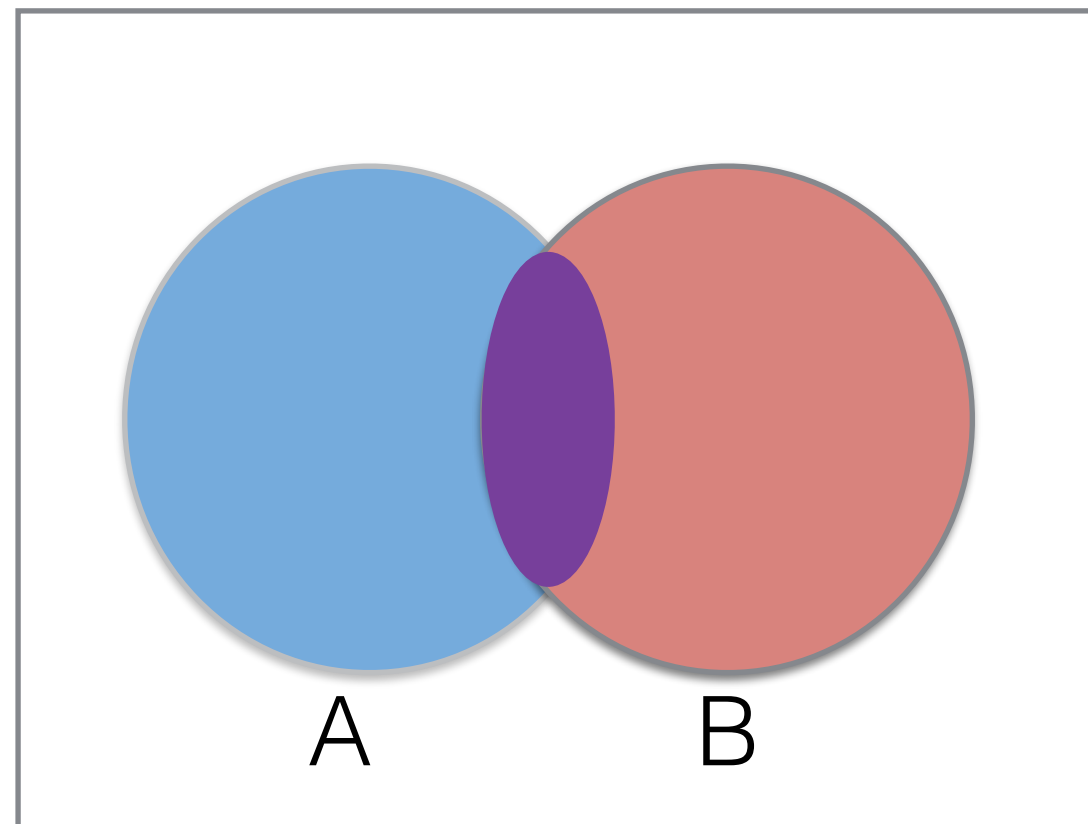


Set Operations



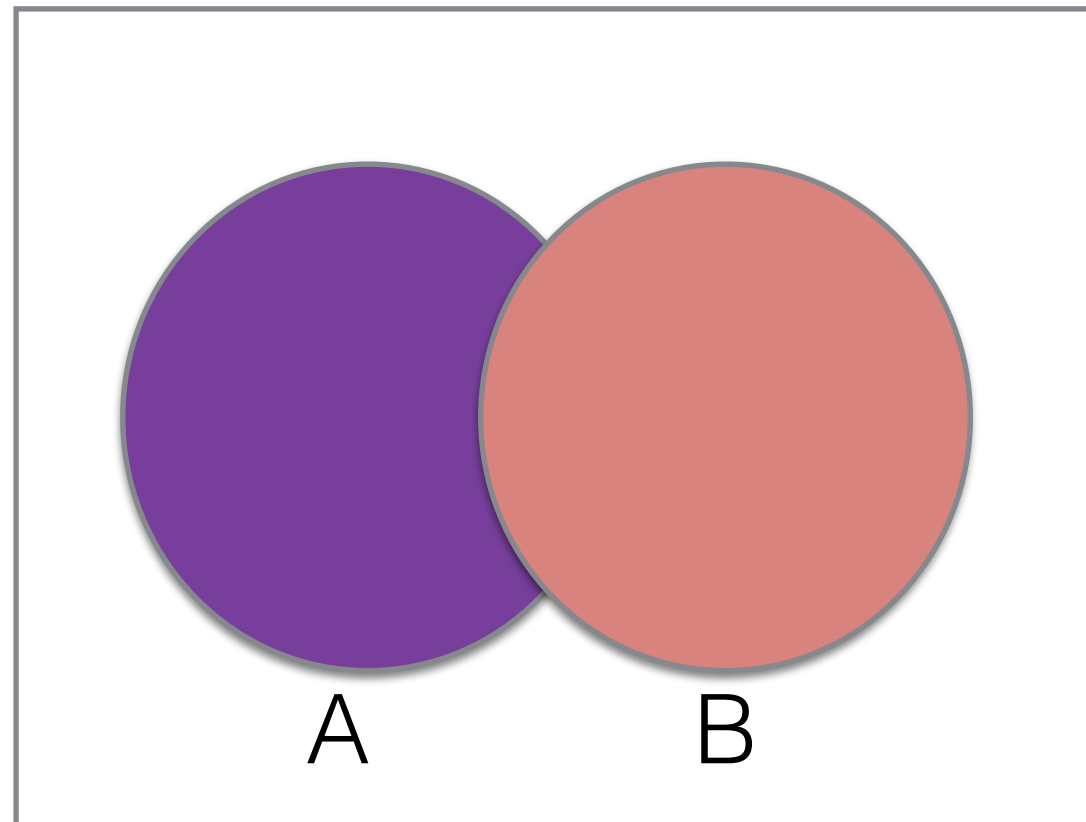
Union: $A + B$

Set Operations



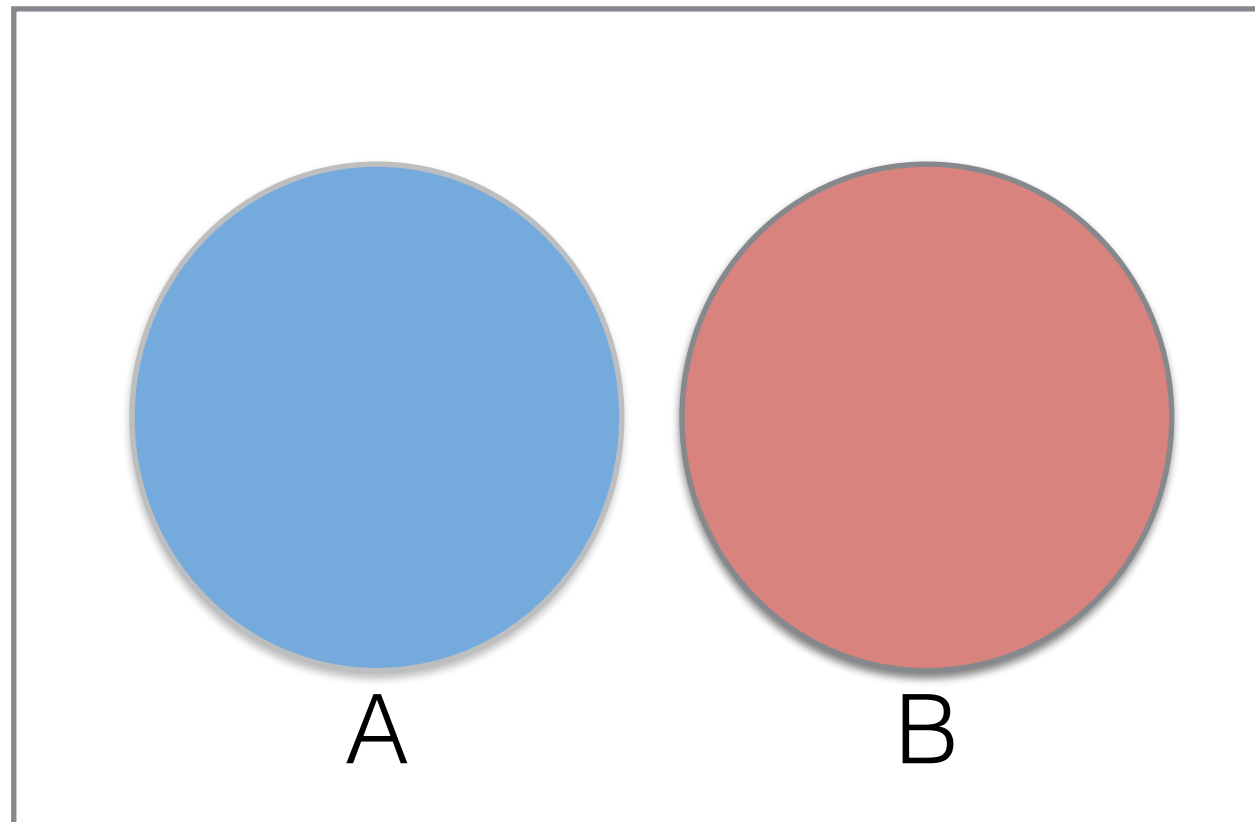
Intersection: $A \cap B$

Set Operations



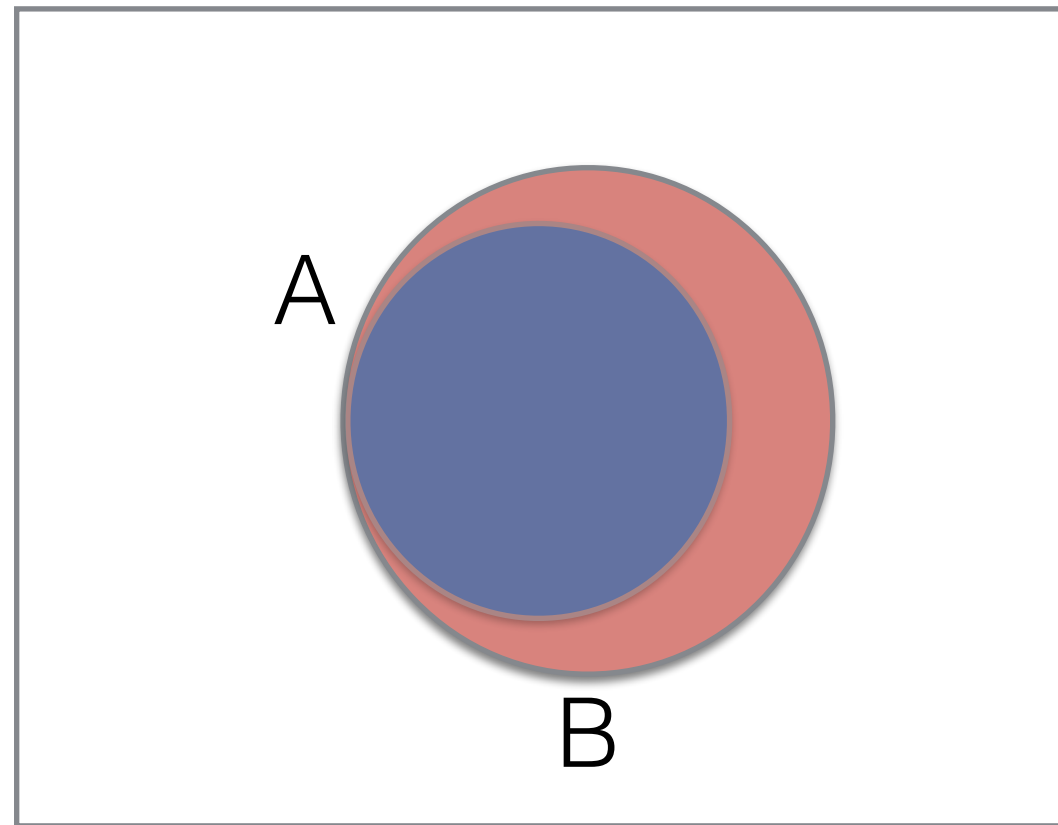
Difference: $A - B$

Set Relationships



Disjoint: no A & B

Set Relationships



Subset: $A \subset B$

Sets

x is a member of set A

x **in** A

subset

everything in B is also in A

B **in** A

singleton

“the set holding just the number 3”

3

cardinality

the number of things in set A

#A

set A is empty

no A

in Alloy, everything is a set,
even single elements

Predicates

Extend propositions with the ability to **quantify** the values of a variable that a proposition is true for

all: Proposition $P(x)$ holds for **all** values of x

all $x \mid P[x]$

all $city \mid \text{Raining}[city]$

all $city : \text{AustralianCities} \mid \text{Raining}[city]$

some: Proposition $P(x)$ holds for **at least one** value of x

some $x \mid P[x]$

some $city \mid \text{not } \text{Raining}[city]$

Quantification

De Morgan's Laws

all $x \mid P[x]$ is equivalent to **not some** $x \mid$ **not** $P[x]$

some $x \mid P[x]$ is equivalent to **not all** $x \mid$ **not** $P[x]$

Alloy Specific Quantifiers

one $x \mid P[x]$

$P(x)$ holds for **exactly** one value x

lone $x \mid P[x]$

$P(x)$ holds for **at most** one value x

none $x \mid P[x]$

$P(x)$ holds for **no** value x

Relations

A proposition that **relates** things together

$=$, $<$, etc.

arity: the number of things the relation relates

$=$, $<$ etc. are all **binary** relations; relate **two** numbers

$3 < 4$, $(5 - 1) = (3 + 1)$, etc.

A relation that relates **three** things together:

$\text{IsSum}(x, y, z) \Leftrightarrow z = x + y$

Relations are just predicates

What Is A Relation?

How would you write down unambiguously
what a relation **means**?

Simplest answer: just list all of the things it relates together.

Sum(x,y,z)

x	y	z
0	0	0
0	1	1
...		
4	12	16
...		
120	-5	115
...		

Each row is a **tuple**: (x, y, z)

Any **relation** is simply a set of tuples.

Relations as Sets

Factor(x, y, z) — when y and z are factors of x

x	y	z
1	1	1
2	1	2
2	2	1
3	1	3
3	3	1
4	1	4
4	2	2
...		

Each row is a **tuple**: (x, y, z)

Any **relation** is simply a set of tuples.

Relations as Sets

$$\text{Factor}(x,y,z) \text{ — } \{(x,y,z) \mid x = y * z\}$$

$$\{(1,1,1), (2,1,2), (2,2,1), (3,1,3), (3,3,1), (4,4,1), (4,2,2), \dots\}$$

Use Factor to define when a number is **prime**,
i.e. define the **predicate** IsPrime[x]

all $y,z \mid (x,y,z) \text{ in Factor}$ **implies** $y = 1$ **or** $z = 1$

Relations as Sets

A relation with **arity** n (i.e. an n -**ary** relation) R ,
is a set of n -tuples of atoms.

A **binary** (2-ary) relation $R(a,b)$ is a set of: **pairs**

A **ternary** (3-ary) relation $R(a,b,c)$ is a set of: **triples**

A **unary** (1-ary) relation $R(a)$ is a set of: **atoms**

i.e. just an ordinary set

Sets are relations, and relations are sets.

Sets are predicates, and predicates are sets.

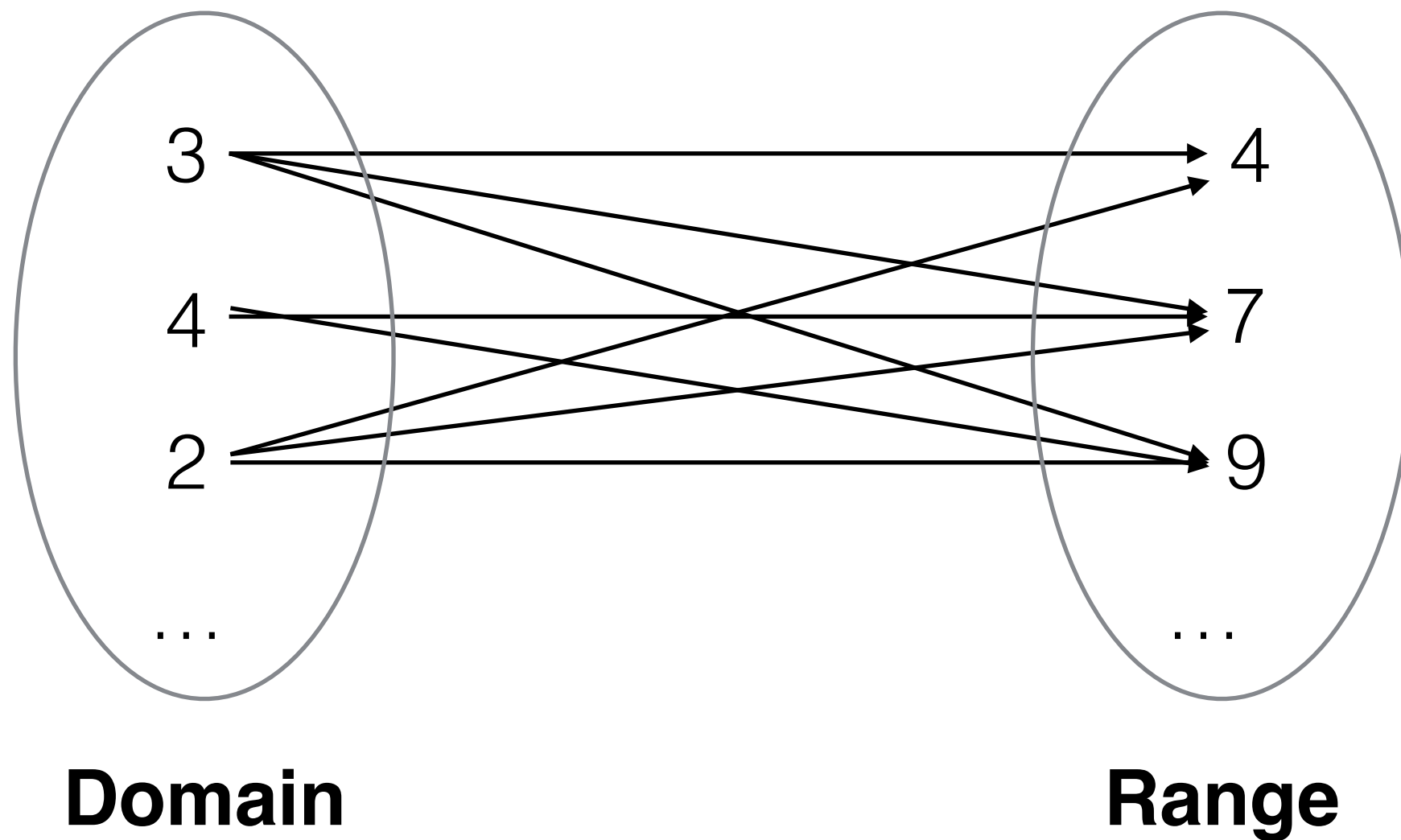
Relations are predicates, and predicates are relations.

Binary Relations as Pictures



THE UNIVERSITY OF
MELBOURNE

Less Than: $<$



Relation Example

Imagine 2 sets of atoms:

Username = {n1, n2, ...}

Password = {p1, p2, ...}

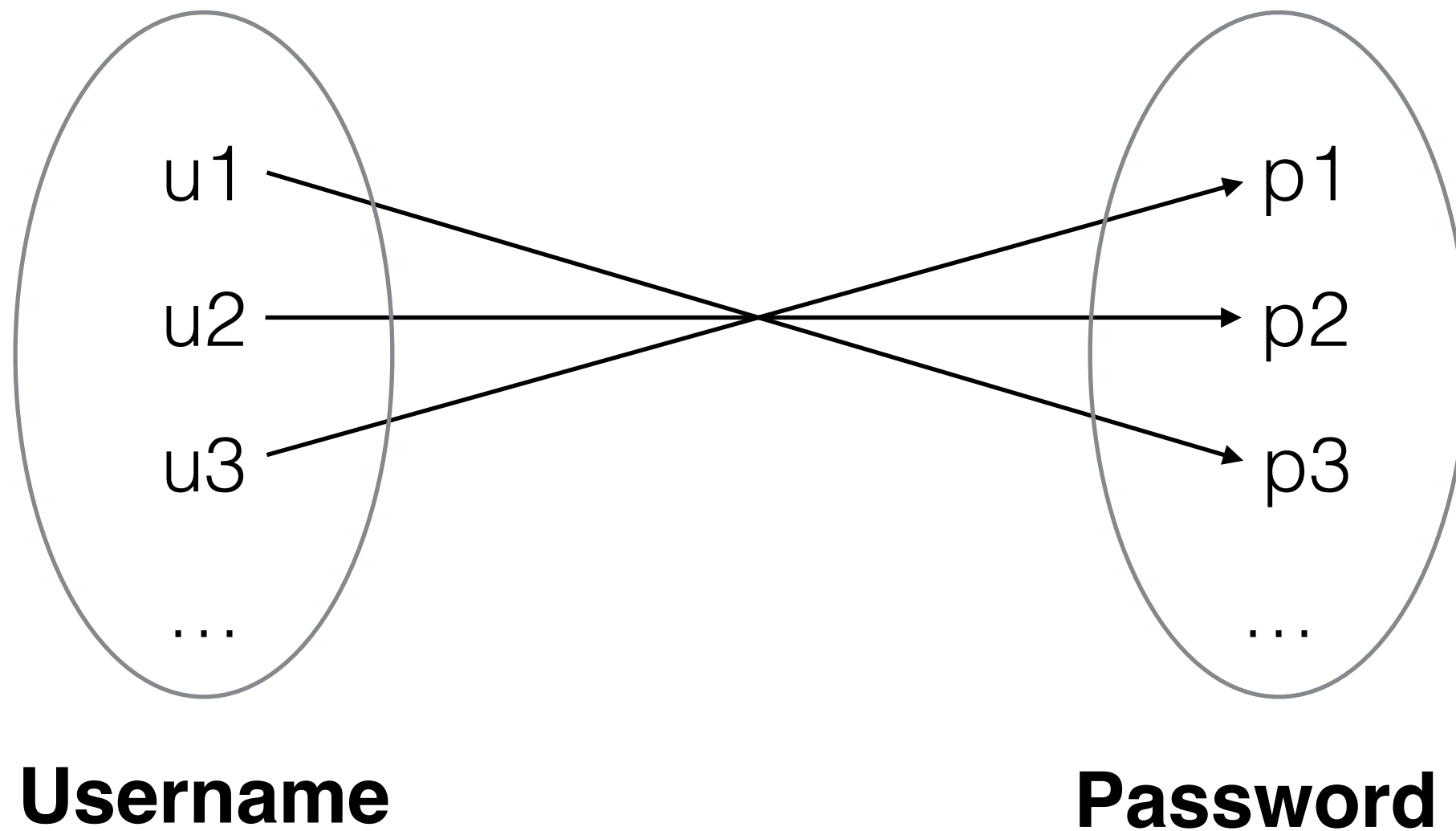
Imagine an LDAP username/password database.

LDAPDB : Username $\rightarrow$ Password

$\text{LDAPDB} \subseteq \mathcal{P}(\text{Username} \times \text{Password})$

Example

Every username has only one password

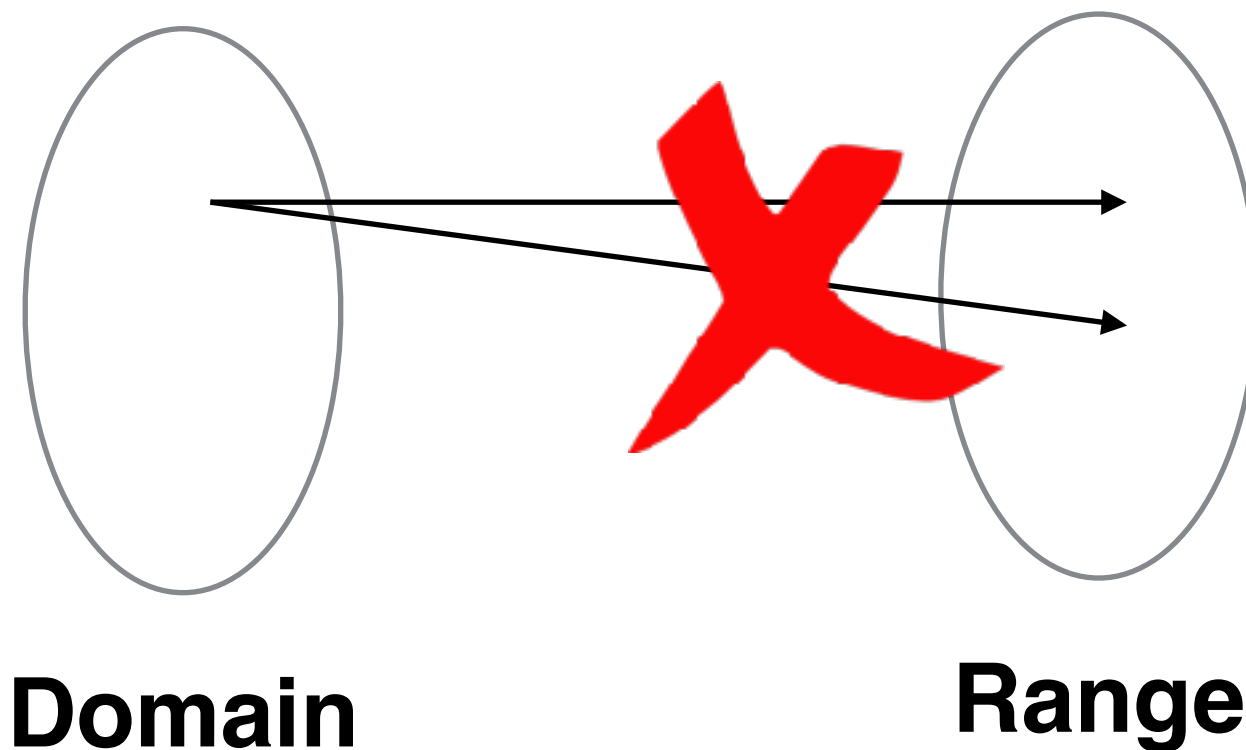


Functions

For every username, there is only one password in LDAPDB

all $u:\text{Username}, pw1:\text{Password}, pw2:\text{Password} \mid$
 $\text{LDAPDB}(u, pw1) \text{ and } \text{LDAPDB}(u, pw2)$
implies $pw1 = pw2$

LDAPDB is a **function**



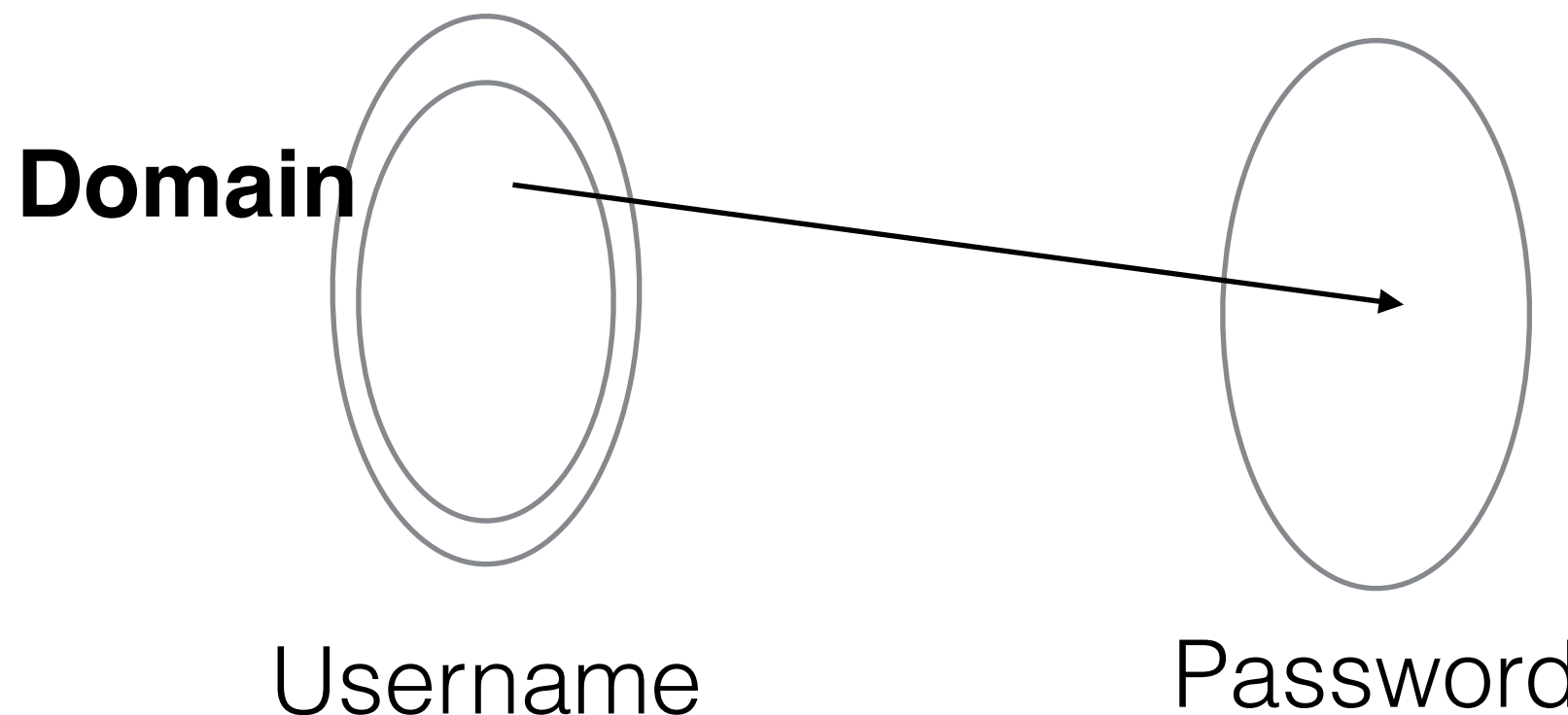
Total vs Partial Functions

LDAPDB : Username $\rightarrow$ Password

Does every username in Username have a password?

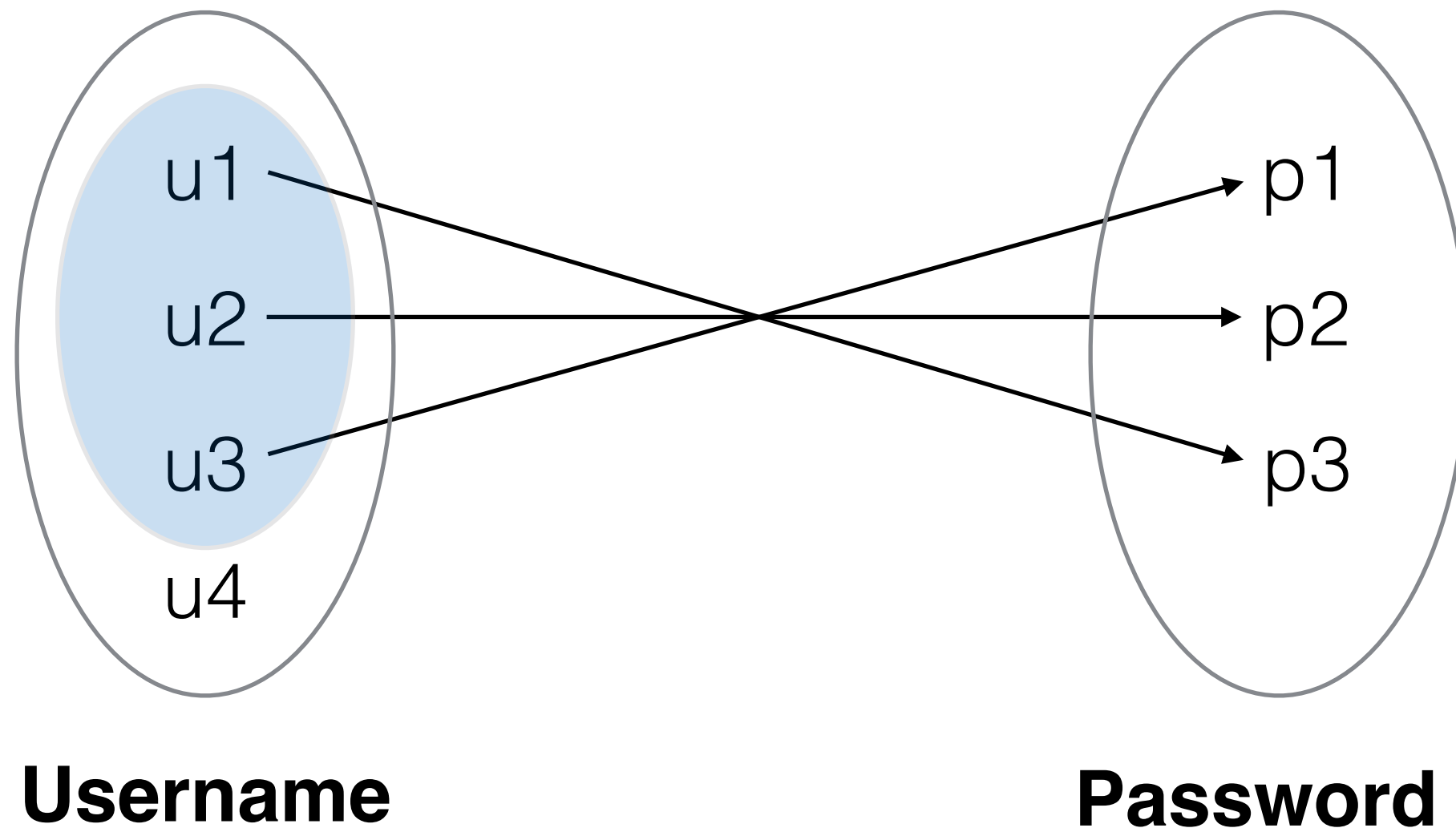
A **total** function gives an answer for every
(type-correct) argument

A **partial** function is one that is not total.



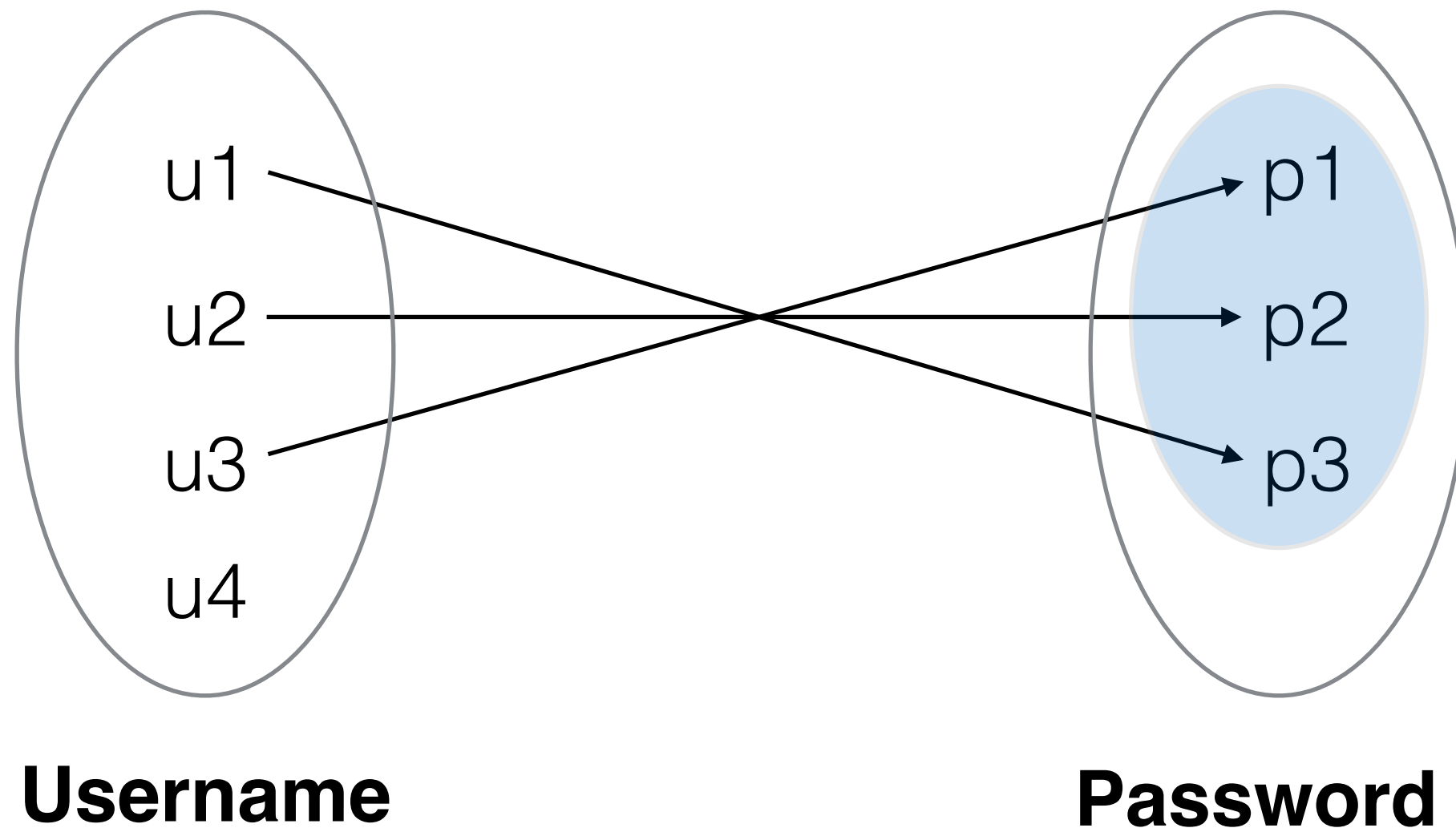
Relation Operations

domain(LDAPDB)



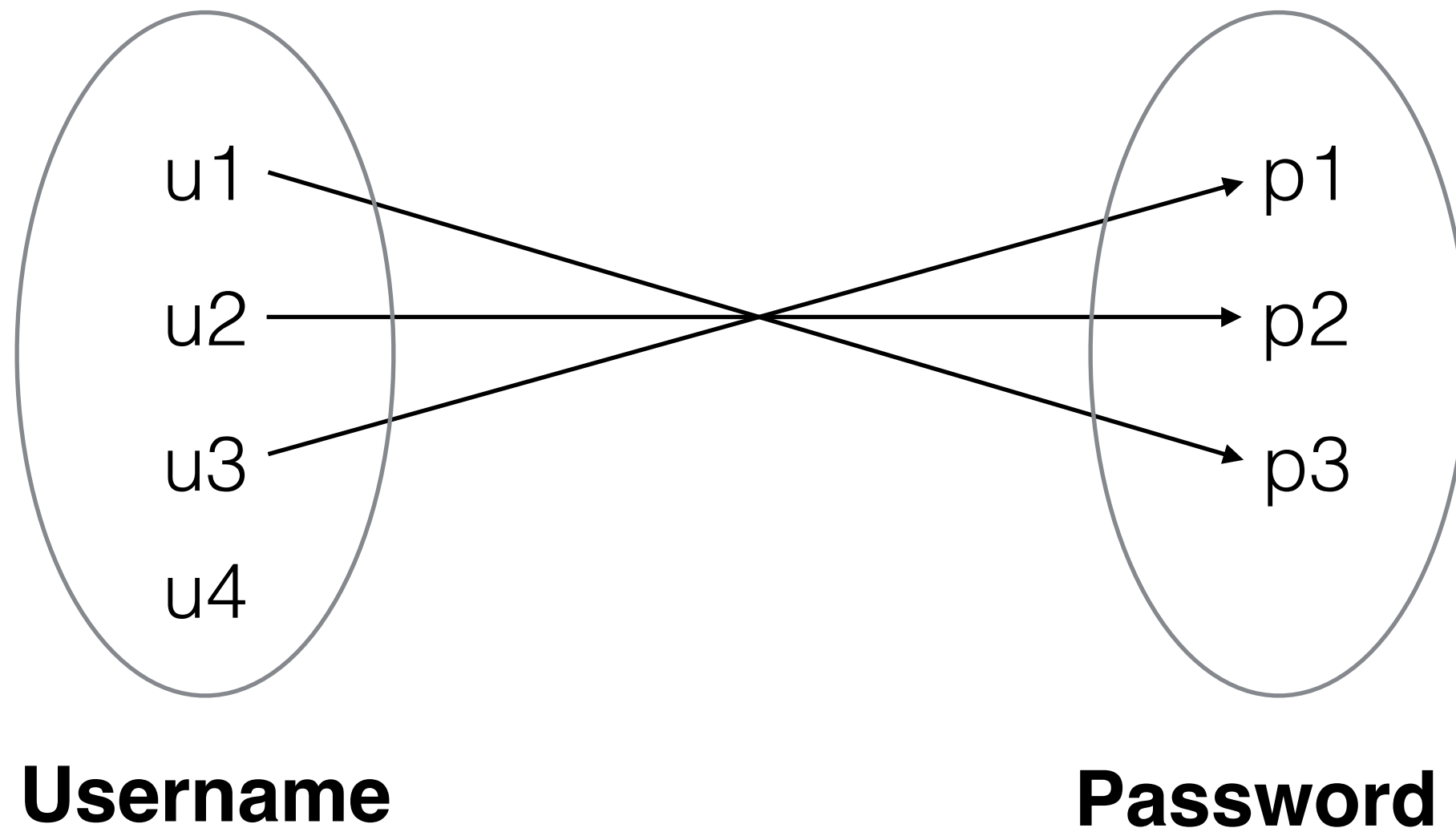
Relation Operations

range(LDAPDB)



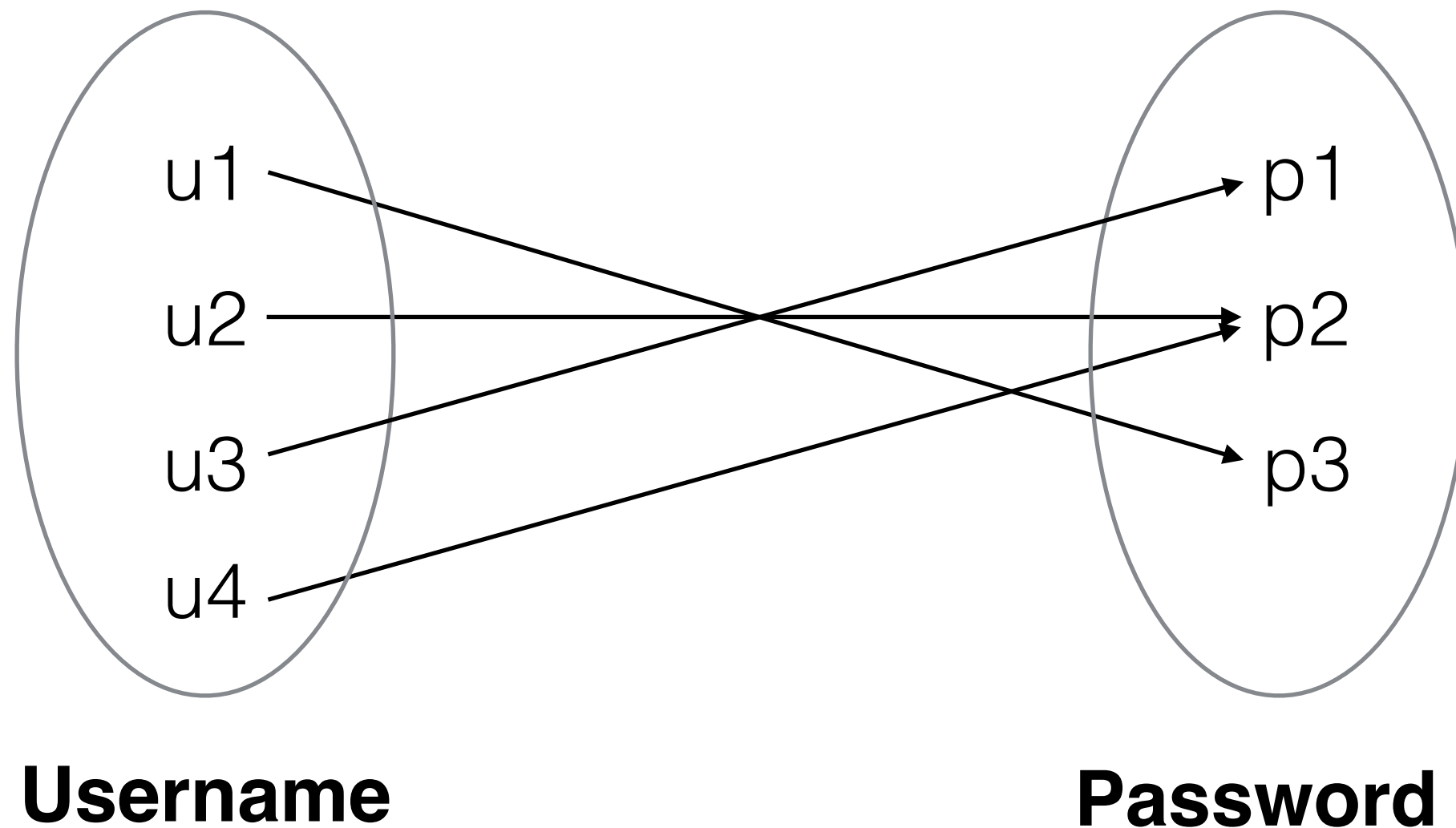
Relation Operations

LDAPDB + (u4 -> p2)



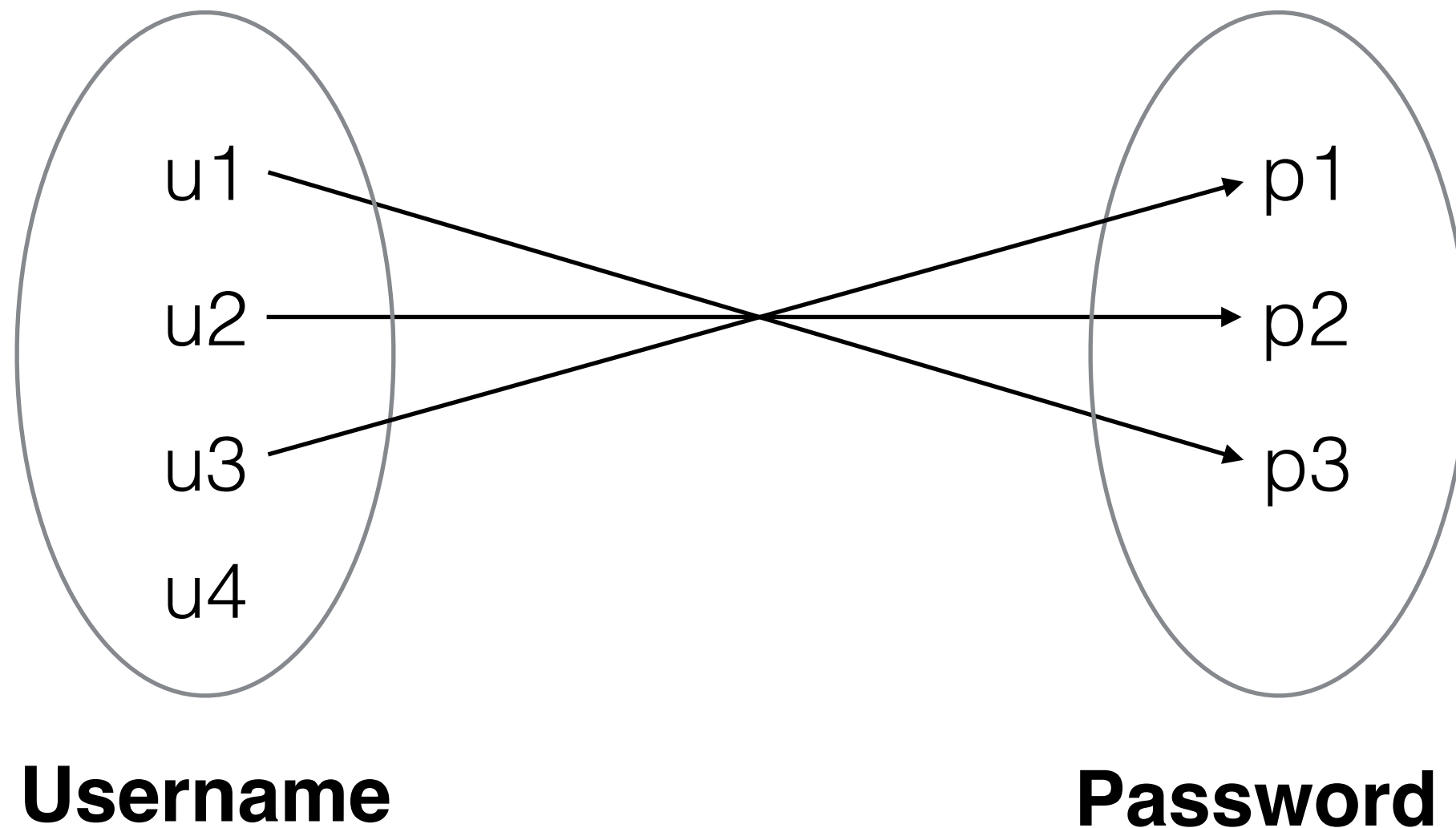
Relation Operations

LDAPDB + (u4 -> p2)



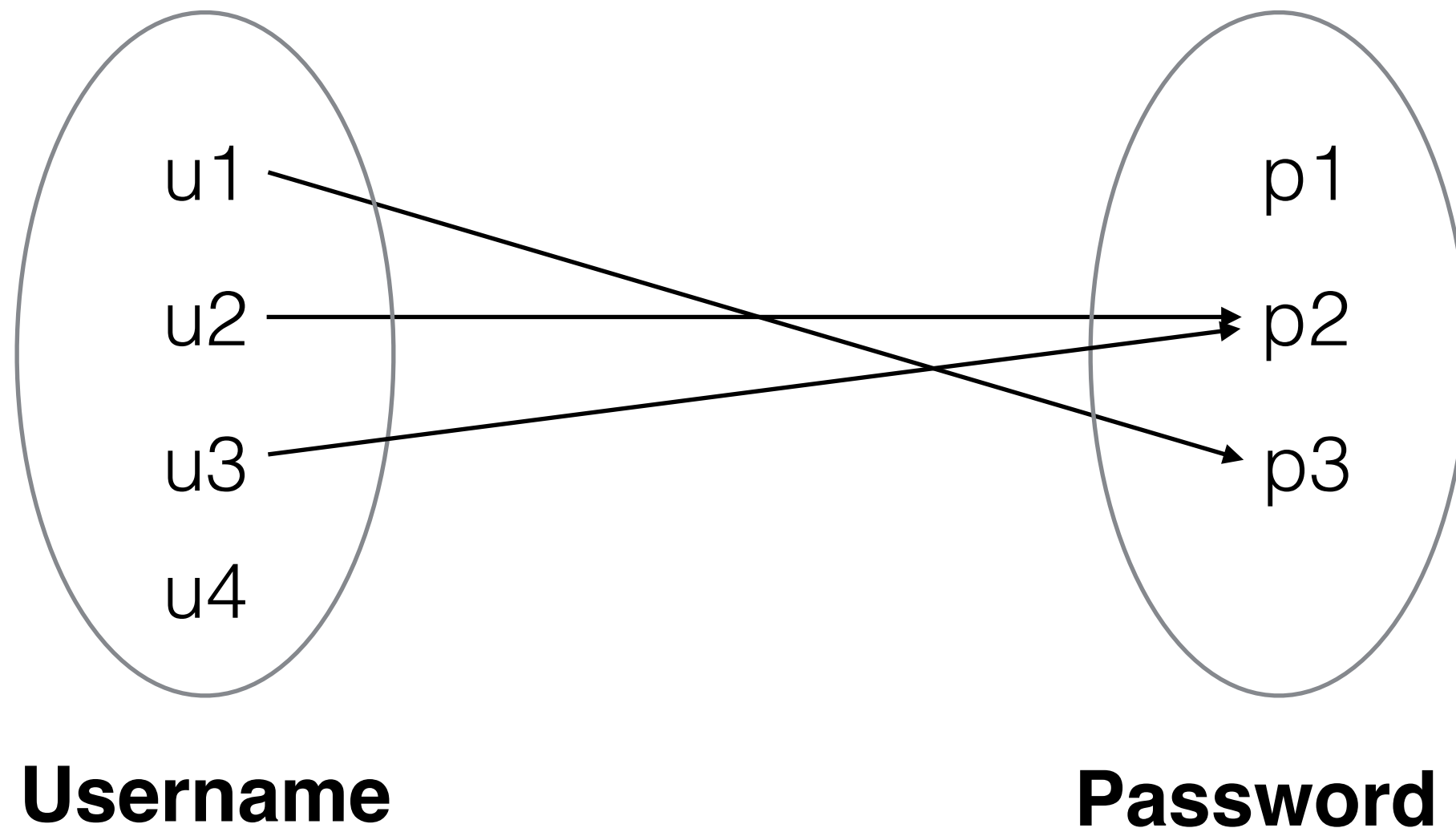
Relation Operations

LDAPDB ++ (u3 -> p2)



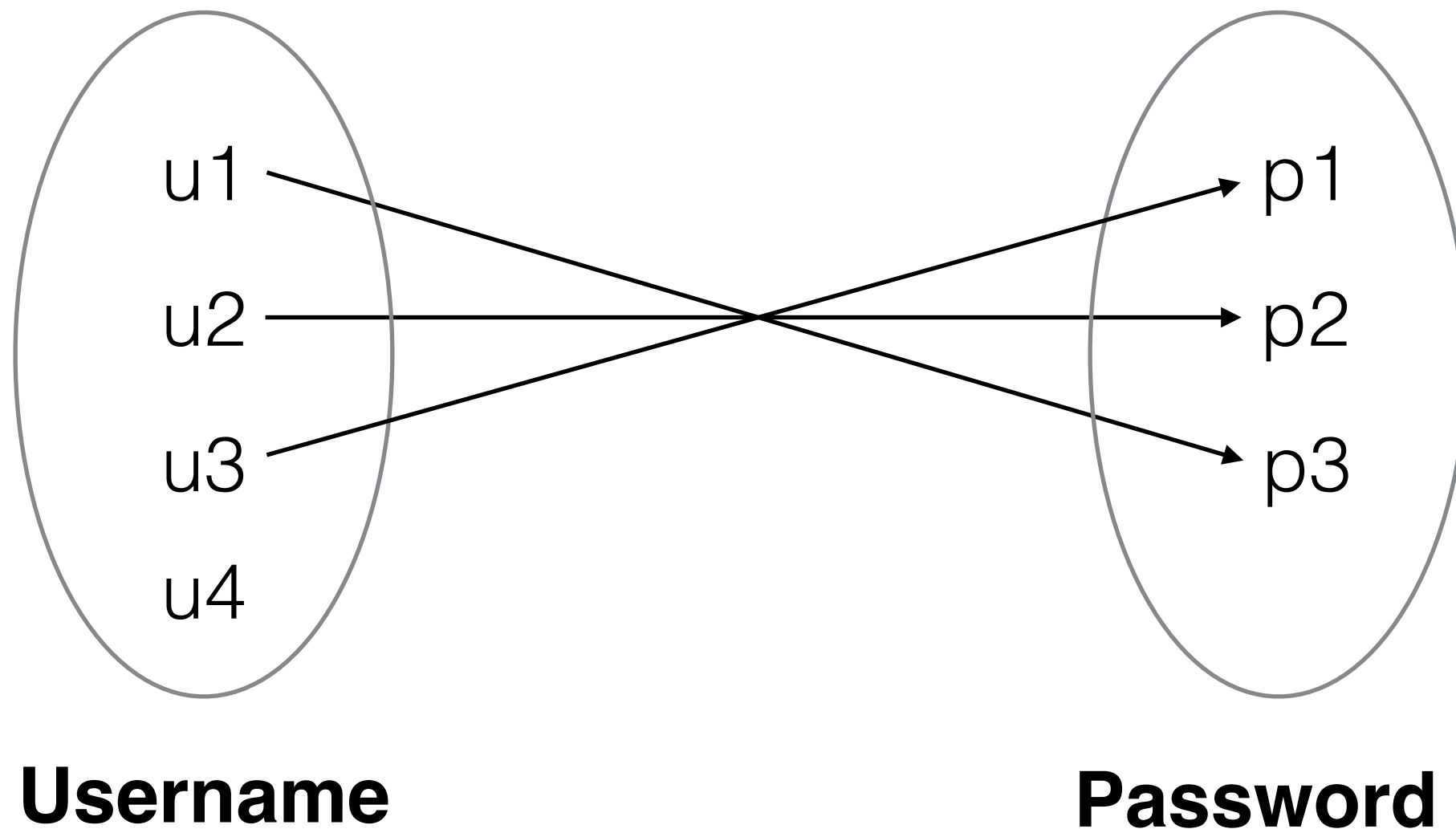
Relation Operations

LDAPDB ++ (u3 -> p2)



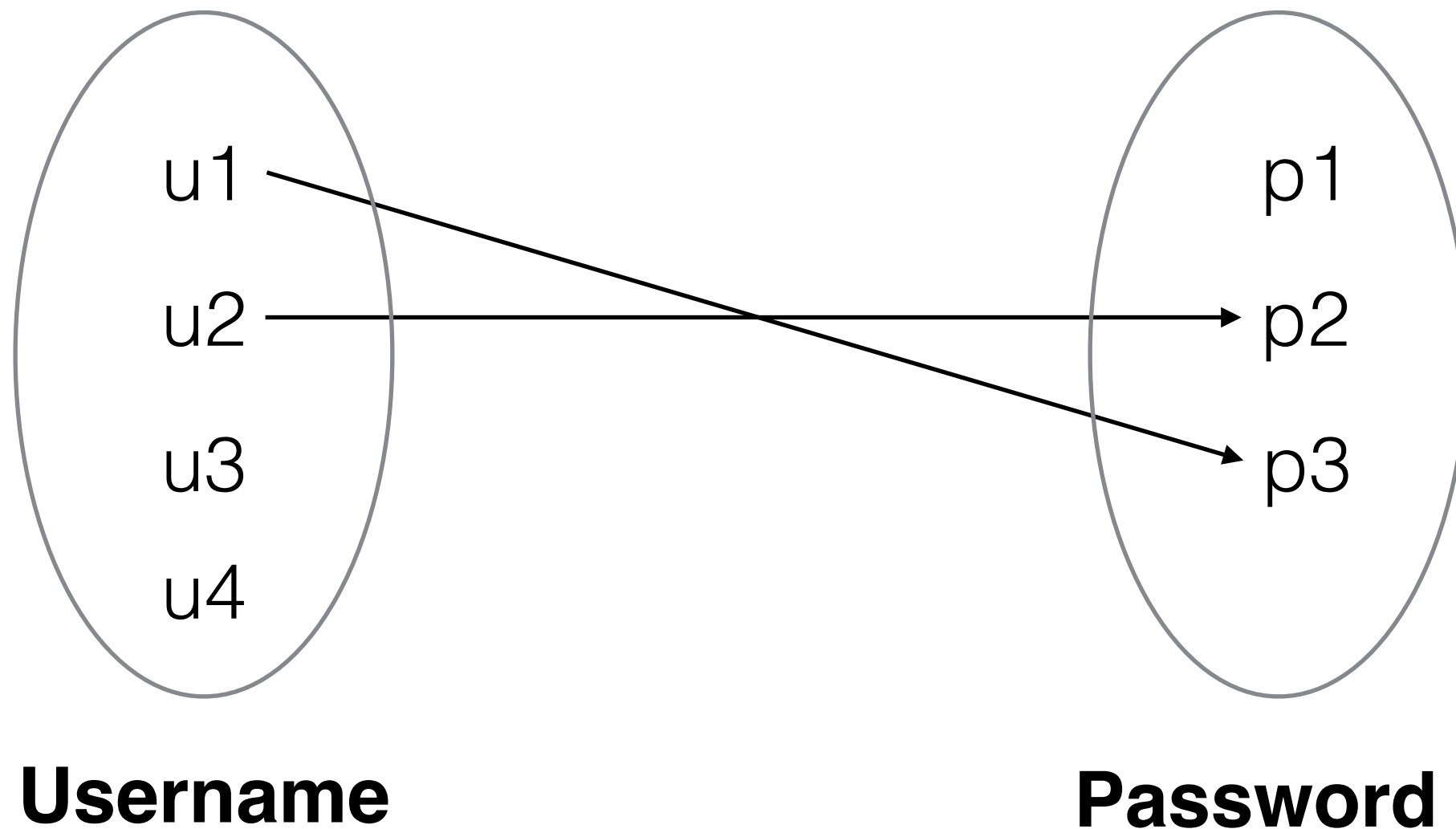
Relation Operations

$\{u1, u2\} \leq: \text{LDAPDB}$



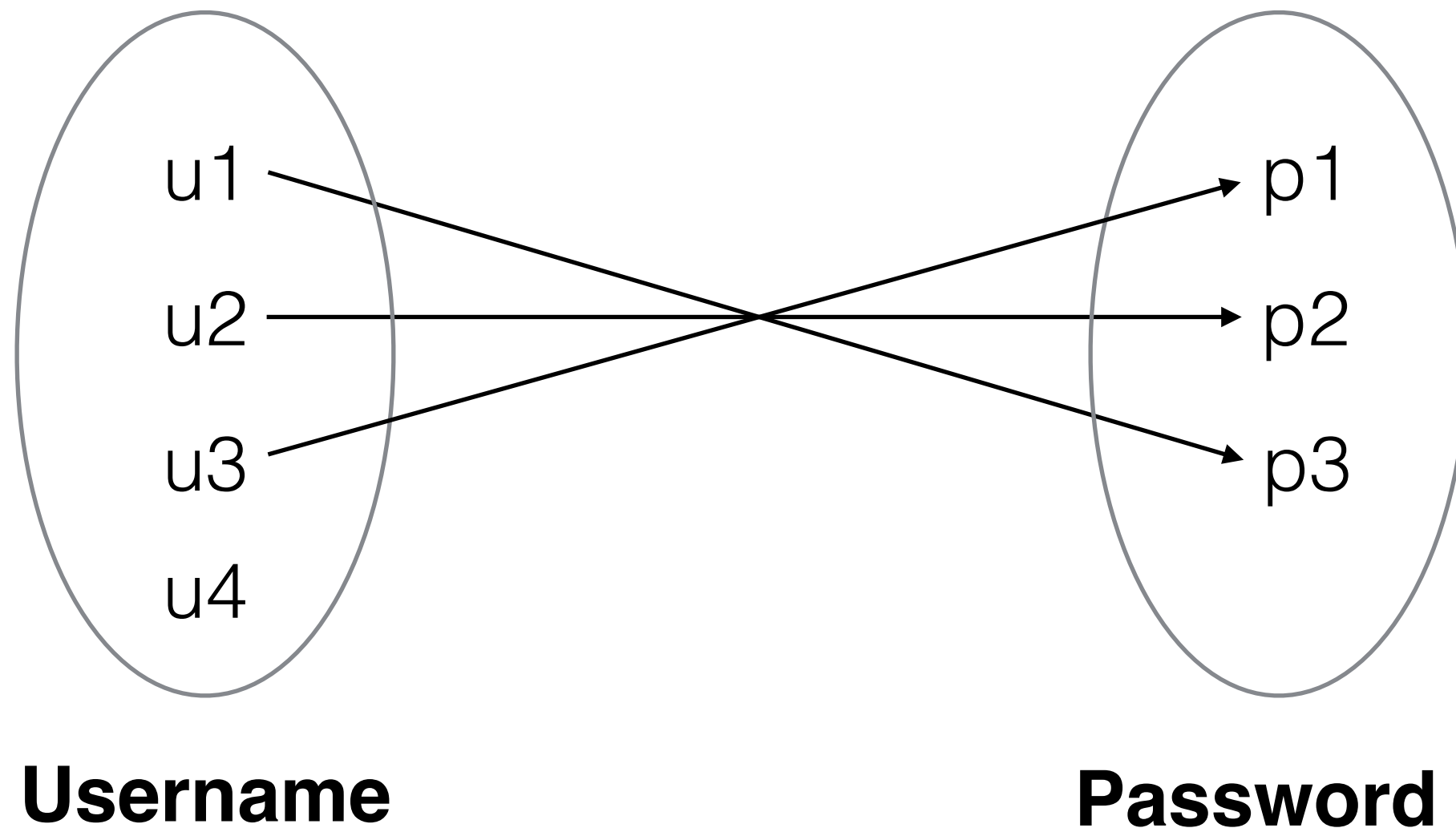
Relation Operations

$\{u1, u2\} \leq: \text{LDAPDB}$



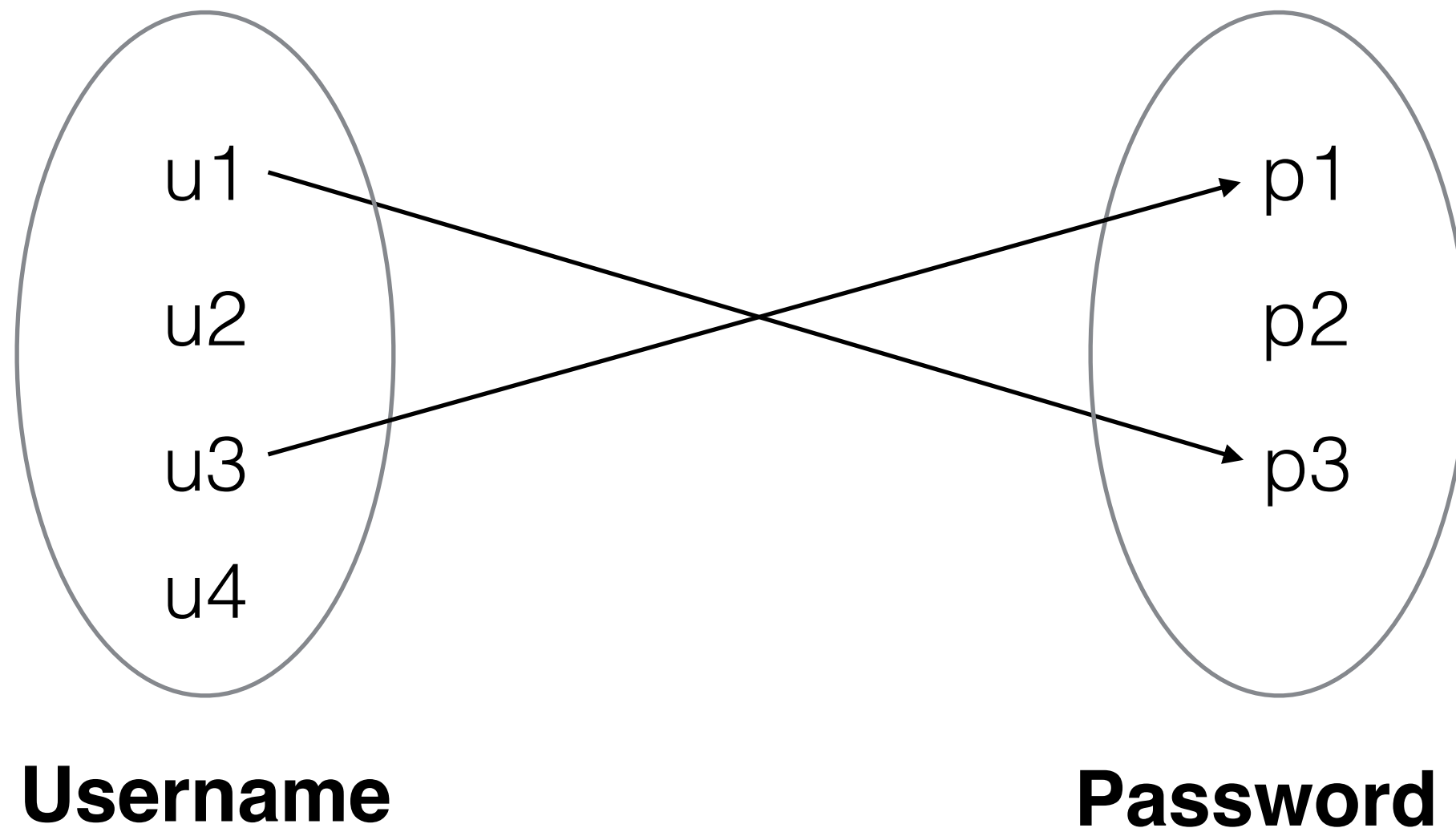
Relation Operations

LDAPDB $\Rightarrow$ {p1,p3}



Relation Operations

LDAPDB $\Rightarrow$ {p1,p3}



Relation Joins



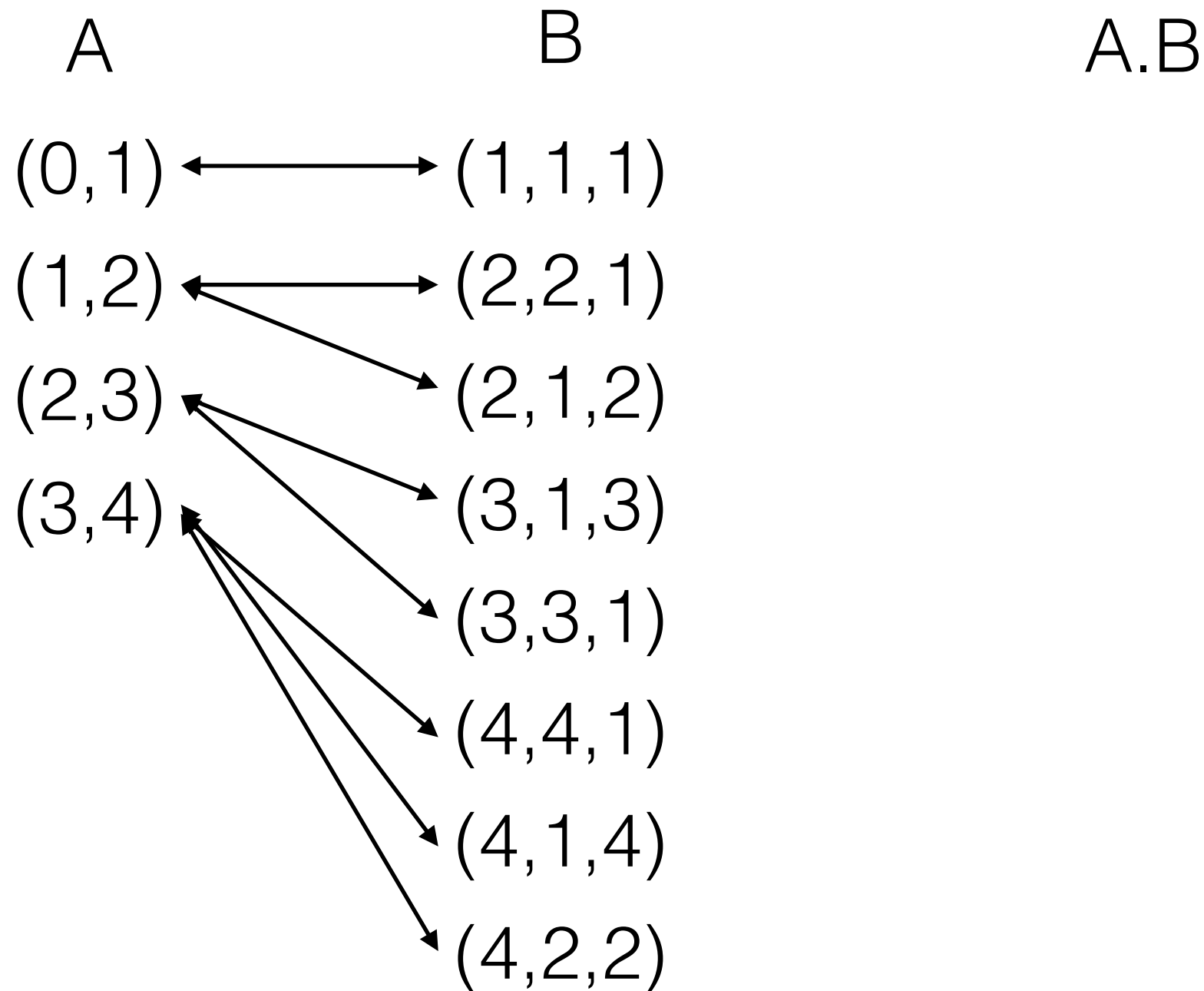
Consider two relations (as sets)

A	B	A.B
(0,1)	(1,1,1)	
(1,2)	(2,2,1)	
(2,3)	(2,1,2)	
(3,4)	(3,1,3)	
	(3,3,1)	
	(4,4,1)	
	(4,1,4)	
	(4,2,2)	

Relation Joins



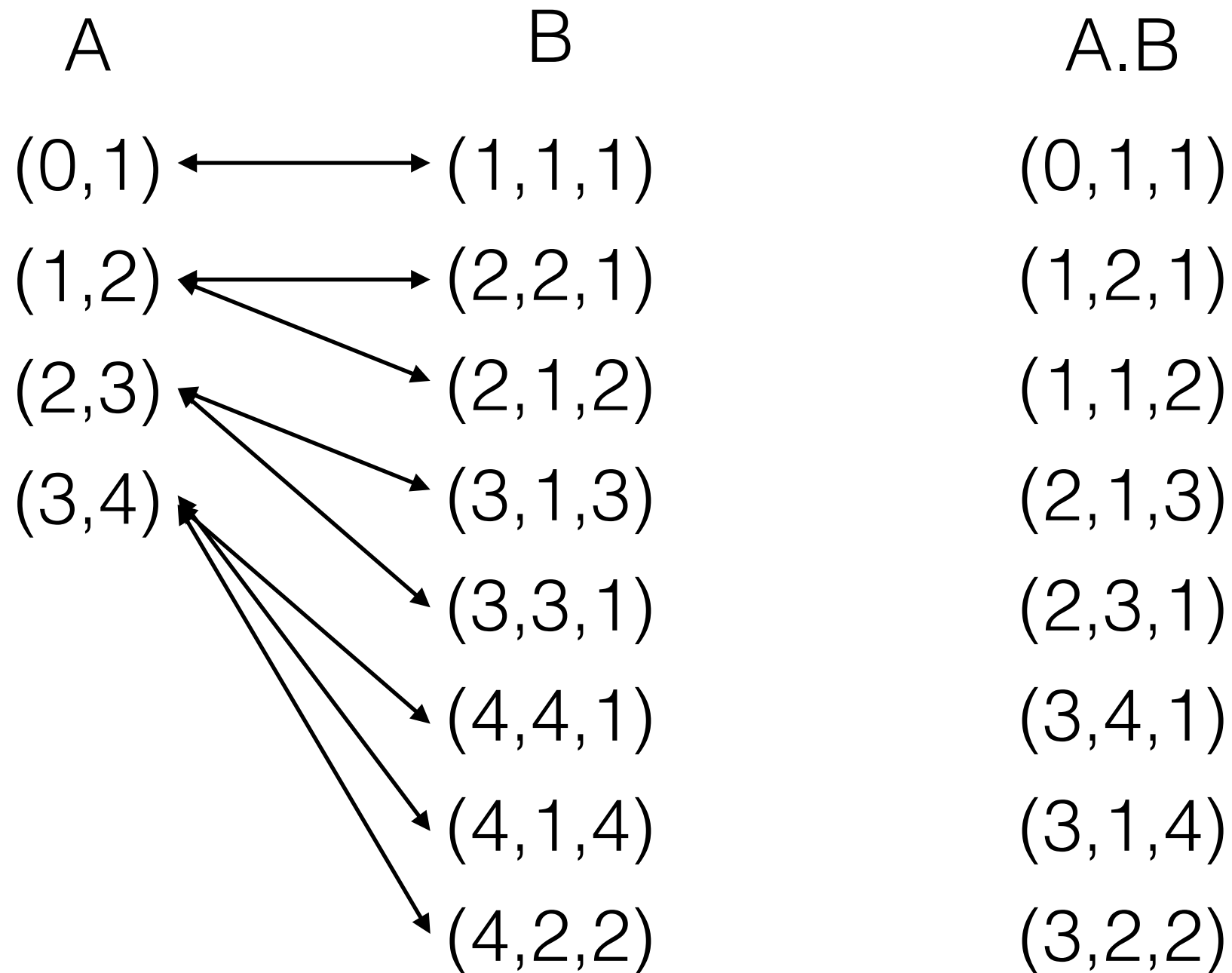
Consider two relations (as sets)



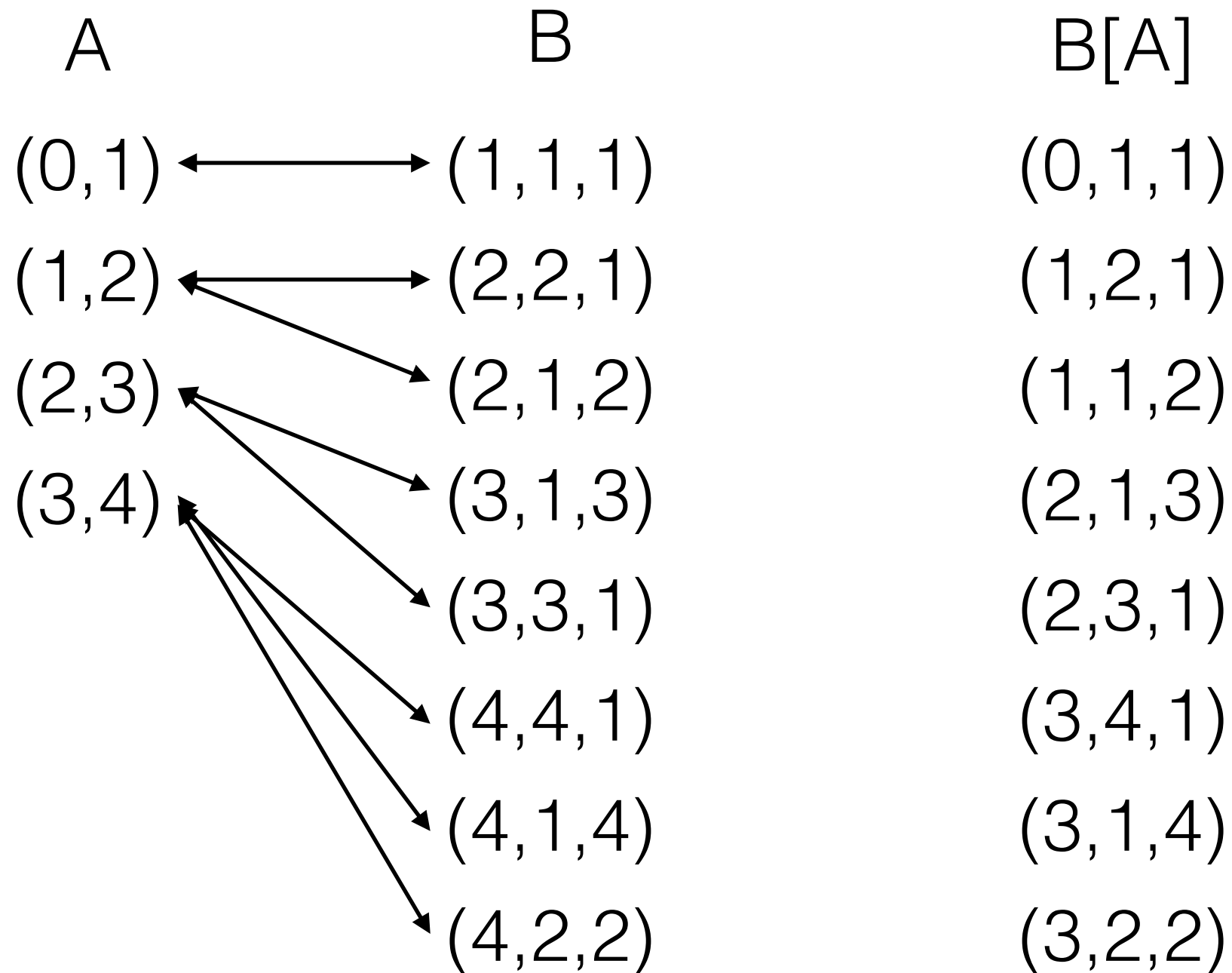
Relation Joins



Consider two relations (as sets)



Relation Joins

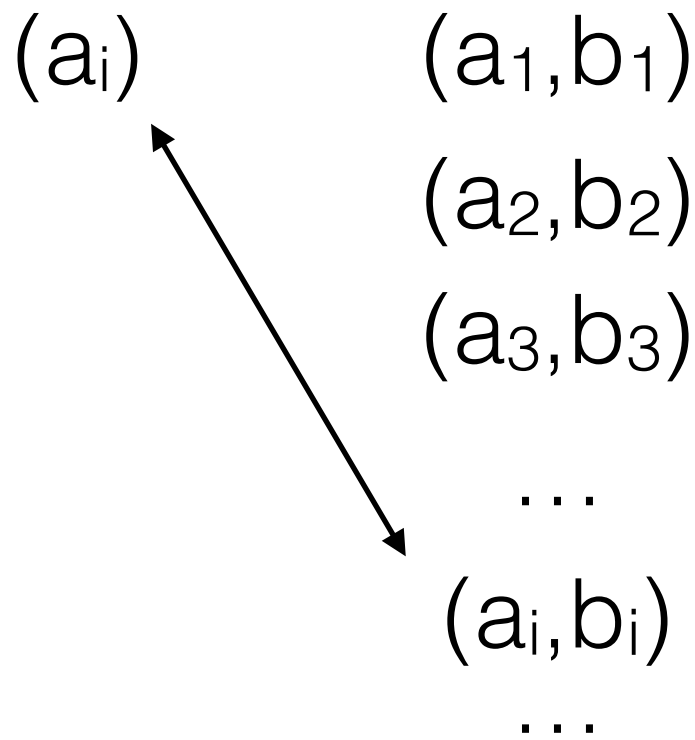


Functions as Relations

Suppose we have a function $f : A \rightarrow B$

f is just a relation

i.e. a set of pairs: (a, b)



Consider some $a_i : A$

What is $f[a_i]$?

What is a_i ? $\{(a_i)\}$

$f[a_i]$ is equivalent to $a_i.f$

i.e. $f[a_i] = b_i$ which is $\{(b_i)\}$

Where Next?

How to use this to model in
software engineering.

Why do that?

Advantages and disadvantages.

How to do this in Alloy and
show properties of models.